

TECHNICAL DEEP DIVE

Engineering the Queue Tracker Ecosystem

React 19

Node.js 24

MongoDB Atlas

Socket.io

Tailwind 4.0

AES-256-GCM

v4.8.2-stable | Build: SHA-4ebba13

/>System Architecture

State Synchronization Pattern

The app utilizes a **Server-Authoritative State** model with optimistic UI updates. Every mutation follows a strict unidirectional flow:

UI Action → Socket Emit → Controller Mutation → DB Persistence → Broadcast Diff →
React Re-render

Networking

- **WSS**: Persistent duplex connection for sub-100ms latency updates.
- **REST**: Stateless endpoints for auth, bulk exports, and reports.
- **Heartbeats**: Custom ping mechanism to track user "Last Online" status accurately.

Database Persistence

- **BSON Structure**: Optimized for rapid reads of performance metrics.
- **Indexing**: Compound indexes on (userId, date) for fast daily metric lookups.
- **TTL Indices**: Automatic cleanup of ephemeral session data.

/>Security Implementation

Zero-Trust approach to identity and data integrity.

1. Secrets Encryption (AES-256-GCM)

```
const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
let encrypted = cipher.update(secret, 'utf8', 'hex');
encrypted += cipher.final('hex');
const authTag = cipher.getAuthTag().toString('hex');
// Result: iv:authTag:encrypted_payload
```

Used for storing TOTP 2FA secrets and sensitive vendor configuration in MongoDB.

2. Authentication

- **JWT Strategy:** HTTP-Only, Secure cookies to mitigate XSS/CSRF risk.
- **Bcrypt Scaling:** Passwords hashed with cost factor 12.
- **2FA:** RFC 6238 compliant TOTP.

3. Authorization

- **RBAC:** Role-Based Access Control via middleware.
- **Revocation:** Force-kick sessions by blacklisting JTI in real-time memory.

/>Real-Time Sync Engine

Handling 100+ concurrent mutations per minute without race conditions.

```
// Server-Side Socket Logic
io.on('connection', (socket) => {
  socket.on('update_stats', async (data) => {
    const updated = await Stats.findOneAndUpdate(
      { userId: data.uid, date: IST_DATE },
      { $inc: { [data.type]: data.change } },
      { new: true, upsert: true }
    );
    // Selective Broadcast to avoid payload bloat
    io.emit('stats_sync', { uid: data.uid, stats: updated });
  });
});
```

Conflict Resolution

Utilizes MongoDB \$inc operator for atomic increments. This ensures that simultaneous "+1" clicks from multiple sources result in the correct aggregate sum.

Bandwidth Optimization

Delta-Sync strategy: Only the changed field is broadcasted, not the entire user object or state tree.

/>Database Modeling

Schema design focused on relational-style integrity in a NoSQL environment.

Core Entities

Users

- `_id`: UUID
- `role`: Enum(Admin, Lead, Agent)
- `twoFactorSecret`: Encrypted AES
- `lastActive`: Date

Stats (Daily)

- `userId`: Ref
- `date`: Indexed ISO string
- `incidents/tasks/calls`: Number
- `comments`: String

Audit & Logging

Capped Collection or Time-Series collection (depending on traffic) used for high-frequency audit logs to ensure sub-millisecond write latency.

>Frontend Architecture

High-Density Grid

Utilizes **Tailwind 4.0 CSS Grid** with sub-grid properties to maintain alignment across 10+ columns without layout shifts during real-time data ingestion.

Performance Hooks

Extensive use of `useMemo` for filtering rosters and `useCallback` for socket emitters to prevent unnecessary shallow-compare re-renders.

State Management

Hybrid approach:

- **Context API:** Global Auth & Role status.
- **Local State (React-Query style):** Real-time productivity data.
- **Ref Hook:** Socket instance persistency across component lifecycles.

/>Timezone & Roster Logic

Solving the "Date Leak" problem for global distributed teams.

Normalization Algorithm

```
function getISTDate() {  
  return new Date().toLocaleString('en-US', {  
    timeZone: 'Asia/Kolkata'  
  }).split(',')[0];  
}
```

The server enforces **Asia/Kolkata (IST)** as the global baseline. All database entries are mapped to this date string, ensuring unified reporting regardless of the agent's actual location.

Smart QH Rotation

The Roster system detects "Current Shift" handlers by intersecting current server time with the defined shift windows (e.g., 6AM-3PM), allowing for dynamic QH highlighting.

/>Infrastructure & Ops

CI/CD Pipeline

- **Vite Build:** Module-level code-splitting.
- **Production:** PM2 Process Manager for clustering.
- **Reports:** Puppeteer Cluster for concurrent PDF generation.

Observability

- **Server Logs:** Structured JSON logging for ELK compatibility.
- **Client Logs:** Remote error reporting for production debugging.

End of Technical Specification